CHECKING THE CHECKS: BEHAVIORAL SCREENS FOR LLM JUDGES OF FORMAL SPECIFICATIONS

Anonymous authors
Paper under double-blind review

ABSTRACT

When AI agents write the trust artifacts for their own code, such as tests and formal specifications, those artifacts need checking at scale. In end-to-end autoformalization the agent chooses its own theorem statements, so a language-model judge must compare the meaning of candidate and reference specifications, and every evaluation built on it inherits its errors. The standard evidence that such a judge can be trusted is agreement with human ratings, collected again for every new model and prompt. We ask whether label-free behavioral properties, scripted edits of a gold reference whose required score response is fixed by construction, predict that agreement for judges they were not selected on. We score 56 judge configurations on 28,591 Lean-validated probe cells, split them by judge model into train, validation and test sets, select a screen on the first two and test it once on the third. One component transfers: the fall in score as theorem statements are deleted from the reference predicts held-out judges' agreement with expert ratings (Spearman 0.56 $[0.08, 0.76]$, one-sided permutation $p = 0.013$, 16 configurations). The association shrinks without a deliberately lexical prompt (0.45) and reverses within the ten fuller-context prompts (-0.37), and our two pre-registered indices do not transfer (-0.33 and -0.27). Choosing a judge by the screen does not beat a fixed human-mirror prompt (0.77 versus 0.85, paired difference -0.09 $[-0.31, 0.06]$), a semantic screen passes 15 of 56 configurations, and a score-optimizing attacker inflates attacked judges and validated reference alike. Label-free screens can rule judges out and expose failures that human agreement hides, but they do not yet replace a human panel for choosing among the judges one would deploy.

1 INTRODUCTION

AI agents now write the trust artifacts for their own code: the tests (Haque et al., 2026) and formal specifications (Yang et al., 2024) on which trust in that code rests. A formal specification states the guarantees a program should satisfy, such as returning a sorted permutation of its input. To trust an AI-written trust artifact, one must check the artifact and check whatever judge performs that check. We start with formal specifications because the proof assistant fixes what each theorem statement means, leaving one question: whether the statements capture the intent.

In end-to-end autoformalization, an agent receives source code and writes the Lean 4 implementation, tests, theorem statements, and proofs without a scaffold (Miranda et al., 2026). Correct solutions take many forms, because none need share the reference's interfaces, data representations, or decomposition into theorem statements. Two questions then decide whether a solution is right: do its proofs establish its own theorem statements, and do those statements express the reference's guarantees? Lean settles the first. It cannot settle the second, because a proof establishes the theorem that was stated, not that the stated theorem covers the intent (Lahiri, 2024). Rerunning the reference's tests and theorems cannot settle it either: they fail against a different representation whether or not the solution is correct (Miranda et al., 2026). What remains is a comparison of meaning between the candidate's and the reference's specifications. Large language models (LLMs) already make that comparison at scale (Miranda et al., 2026; Lin et al., 2025). Every evaluation built on such a judge inherits its errors, so the judge's reliability decides what the benchmark measures.

How do we know that such a judge can be trusted? The standard evidence is agreement with human ratings: a judge counts as reliable once its scores correlate with expert annotations (Chen

et al., 2026) or rubric-based ratings (Liu et al., 2023; Zheng et al., 2023). Agreement varies across models, tasks, and rated properties, so evidence for one judge configuration does not carry over to the next (Bavaresco et al., 2025). The check is therefore repeated for every new model and prompt, and each repetition costs expert ratings. Reusing one rated set is cheaper, but choosing among many configurations against the same ratings risks adaptive overfitting (Dwork et al., 2015). On our own earlier panel, the best of eight configurations chosen on half of the problems overstates its agreement on the other half by 0.15 in Spearman correlation (Appendix C.6).

Label-free behavioral properties offer a way out. A property fixes, by construction, how a judge's score must respond when a scripted edit preserves or damages a gold reference: a copy must score the maximum, deleting theorem statements must not raise the score, and renaming a bound variable must not change it. Checking these responses costs judge calls, not ratings. An earlier version of this paper showed that such properties track human agreement on the judges they were tuned on, which is not evidence that they transfer. Here we make transfer the question: we select a screen on one set of judge models, check it on a second, and test it once on a third that neither step touched.

We instantiate this test for Lean 4 formal specifications judged against a gold reference (Section 4). Fifty-six judge configurations, six models with nine prompts in a single-call harness and in the Terminus2 agent harness, score 28,591 Lean-validated probe cells: identity and empty candidates, cumulative deletion and bug ladders built from the gold references, rewrites of candidates, and repeated calls. Each configuration's agreement with the mean of three expert ratings on 45 held-out VeriBench candidates is the criterion. A pre-registered family of 22 indices, which includes the two indices this paper previously proposed, is ranked on the train judges, chosen on the validation judges, and tested once on the held-out judges under gates fixed before any call.

One component transfers. The fall in score as theorem statements are deleted from the reference, selected on the train and validation judges, predicts the held-out judges' agreement with the experts (Spearman 0.56, 95% interval $[0.08, 0.76]$, one-sided permutation $p = 0.013$, 16 configurations). The association has a wide interval and a scope limit: it shrinks to 0.45 when a deliberately lexical prompt is excluded and reverses to -0.37 within the ten fuller-context prompts. The two pre-registered indices from our earlier work, $\mathrm{TI}_{\mathrm{var}}$ and PTI_{v0}, do not transfer (-0.33 and -0.27). Choosing a judge by the transferred screen does not beat choosing a fixed human-mirror prompt (expected agreement 0.77 versus 0.85, paired difference -0.09 $[-0.31, 0.06]$). Two further studies characterize the judges rather than the screen: a Lean-validated semantic screen that 15 of 56 configurations pass, and a score-optimizing attacker that inflates the attacked judges and the validated reference alike.

The evidence supports a narrower use than our earlier version claimed. Label-free screens can rule judges out and expose failures that human agreement hides; they do not yet replace a human panel for choosing among the judges one would deploy. Our contributions are:

- A pre-registered train, validation and test evaluation of label-free judge screens over 56 configurations, with Lean-validated probes and one sealed test on held-out judge models (Sections 4 and 5.2).
- A positive transfer result for gold deletion monotonicity, reported with its interval and its prompt-group sensitivity, and a negative transfer result for the two pre-registered indices (Section 5.2).
- A selection-utility comparison showing no demonstrated advantage over a fixed human-mirror prompt (Section 5.3).
- Two characterizations of current judges: a semantic screen for vacuous, weakened and over-constrained specifications, and a score-optimizing attack (Sections 5.4 and 5.5).
- A provenance audit and a selection-bias analysis of our earlier 72-pair validation set, whose results move to Appendix C with their caveats.

2 Related Work

Validating LLM judges through human agreement. G-Eval (Liu et al., 2023) and MT-Bench (Zheng et al., 2023) establish a judge's reliability by its agreement with human ratings, so each new judge or task calls for the same comparison. Bavaresco et al. (2025) measure that agreement across 20 tasks and find it task dependent, which is why the comparison cannot be made

once for all judges. Schroeder & Wood-Doughty (2024) measure consistency across repeated judgments, which detects instability but not a consistently wrong judge. We keep human agreement as the criterion, collect it once, and ask whether checks whose required direction is known without ratings predict it for judges they were not selected on.

AI judges for AI-written trust artifacts. ICE-SCORE (Zhuo, 2024) and CODEJUDGE (Tong & Zhang, 2024) score generated code with a language model where execution gives incomplete evidence, and validate the judge by its agreement with human or execution-based labels on their own tasks. SWT-BENCH validates agent-written tests by execution against the developers' fix (Mündler et al., 2024), which requires the test and the reference to share one interface, the condition that end-to-end formalization breaks. For formal statements, FORMALALIGN learns an alignment score between informal and formal statements (Lu et al., 2025), whereas our target is the behavior of a judge comparing candidate and gold specifications. Prover pipelines filter autoformalized statements with an LLM judge (Lin et al., 2025; Wang et al., 2025). REFORM validates its statement judge against expert annotations (Chen et al., 2026), evidence that does not transfer to the next judge configuration. VERIBENCH scores agent-written Lean specifications with an LLM judge (Miranda et al., 2026); we study when such judges can be trusted. Each of these judges answers a question that execution or proof checking cannot settle: whether an artifact means the right thing.

Behavioral tests with a known direction. Metamorphic testing checks a program without an oracle by fixing how its output must change under a transformation of its input (Chen et al., 1998). The test-oracle problem it addresses (Barr et al., 2015) is the one an AI-written trust artifact raises: running the artifact cannot tell whether the artifact is right. CHECKLIST applies the same idea to NLP models with templated invariance and directional tests, and reveals failures in models with high held-out accuracy (Ribeiro et al., 2020). Our properties are metamorphic relations on the judge: identity, ladder monotonicity, and rewrite invariance fix the required response to a scripted edit of a gold artifact. The one-time human panel adds what metamorphic testing alone lacks, a measurement of whether the relations track the human criterion across judges, and the held-out judge split measures whether that association survives on judges the relations were not chosen on.

3 PROBLEM SETUP

Trust artifacts, intent, and formal specification. An item consists of an informal problem statement, a gold Lean 4 artifact g, and a candidate artifact x written by an agent. We call the informal requirement the *intent*. An artifact is a Lean source file with an implementation and, potentially, theorem statements, predicates, unit tests, and proofs; its theorem statements and tests are its trust artifacts. Its *formal specification* is the set of theorem statements about the implementation together with the predicates they use. A sorting specification, for example, can require both ordered output and preservation of the input elements. A proof establishes a theorem under its stated assumptions; it does not determine whether that theorem captures the intent. A unit test checks one concrete case and is not part of the specification in our terminology (Appendix G). We use "specification" in this sense whether an agent, a person, or a script wrote it.

What the judge must decide. The target is *reference-based specification quality*: how completely and correctly the candidate specification captures the guarantees of the gold reference. A candidate can type-check yet omit a requirement, impose an unintended assumption, or assert a vacuous property. The judge prompts ask which reference guarantees the candidate captures and assign zero to trivial statements such as `True` (Appendix A). This instruction does not establish that judges detect every such failure. When a candidate shares the reference's interface, automated proof attempts in both implication directions can sometimes establish equivalence with the reference specification (Lahiri, 2024; Poiroux et al., 2025). Such attempts do not decide equivalence in general, and a failed attempt does not show inequivalence. Across differing representations, these checks need a specified correspondence. BEq+ reports both false positives and false negatives relative to human judgments (Poiroux et al., 2025). We did not benchmark such a tool against the graded reference-coverage ratings studied here. A gold reference gives the judge a concrete comparison target, but it does not make the judge immune to strategic manipulation (Robertson & Koyejo, 2025).

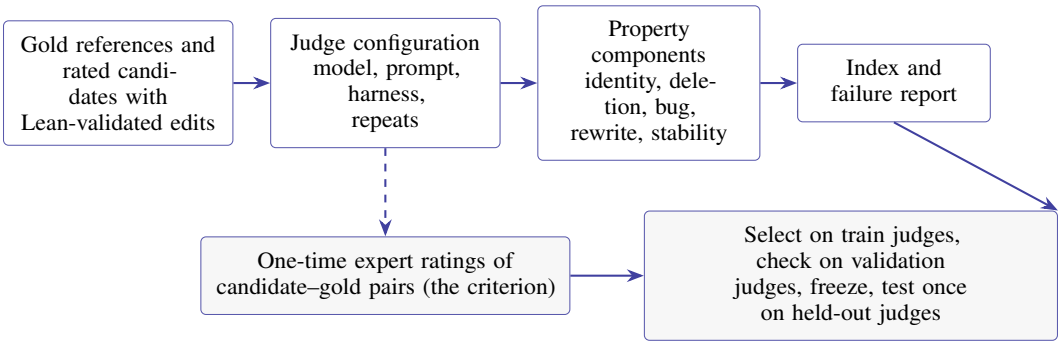


Figure 1: **Human ratings enter once (bottom); every later judge faces only the automated checks (top).** The bottom path selects the screen on the train and validation judges and evaluates the frozen choice once on judge models that neither step touched.

Judge configuration. A judge configuration is $\theta = (\text{model}, \text{prompt}, \text{harness}, n_{\text{repeats}}, \text{aggregation})$ with score $J_\theta(x, g) \in [0, 1]$. Every panel in this paper takes the median over repeated calls. Two prompt families render the inputs differently. The legacy prompts of our earlier version fill a field labelled "REFERENCE IMPLEMENTATION" with the gold file's final theorem block and show the full candidate file. The fuller-context prompts of VERIBENCH's judge show the complete reference, the task's Python source, and the full candidate. The measured task therefore depends on the rendering as well as on the nominal rubric, and a changed rendering requires a new assessment. A harness is either one gateway call that carries the prompt bytes, or the Terminus2 agent harness, which receives the same bytes as its task and writes a verdict file from inside a sandbox.

Two levels of agreement. *Artifact-level agreement* is the correlation between one judge's scores and the human ratings over candidate–gold pairs; on the 45 held-out items of Section 5.1 we call it the *criterion*. *Judge-level association* is the correlation, across judge configurations, between their artifact-level agreement and a label-free component or index. The first says how well one judge tracks the raters; the second says whether the screen orders judges the way the raters would. A screen *transfers* when the judge-level association measured on the judges it was selected on persists on judges it was not.

4 SCREENING JUDGES WITH LABEL-FREE PROPERTIES, TESTED ON HELD-OUT JUDGES

Ratings once, checks for every judge, transfer tested once. Figure 1 and Table 1 lay out the protocol. Experts rate candidate–gold pairs once; these ratings define the criterion of Section 3. For each judge configuration we measure its agreement with the ratings and, separately, its responses to scripted edits of gold references and rated candidates. The edits cost judge calls but no new ratings. Judge configurations are then split by judge model and harness into train, validation, and test sets. The screen is selected on the train judges, checked on the validation judges, frozen, and evaluated once on the held-out test judges under gates fixed before any judge call. A later configuration faces the same frozen checks without access to the ratings.

Probes and components. Every probe is a scripted edit of a gold reference g or of a rated candidate x, and every edited file must elaborate under Lean 4.22 with Mathlib; an edit that fails to elaborate is excluded before any judge call and counted. Ten components summarize a configuration's responses (exact definitions in Appendix B). *Identity* is the median score of each gold against itself, and *blank* is one minus the score of an empty candidate. *Gold deletion monotonicity*, $C_{\text{ms,gold}}$, is the component this paper selects, defined below. *Candidate deletion monotonicity* and *gold bug monotonicity* apply the same ladder construction to a rated candidate and to cumulative implementation bugs in the gold. *Alpha-renaming*, *proof-to-*`sorry`, and *reordering* invariance are one minus the absolute score change under the rewrite, and *spam non-increase* is one minus any score gain after theorem statements are appended to a candidate. *Stability* is one minus twice the population standard deviation across three

Stage	Human ratings	Judge computation
Establish the criterion	Experts rate candidate–gold pairs once	Measure each configuration's agreement and its property components
Select and freeze	Use the train and validation judges' agreement	Rank the candidate family on the train judges, pick on the validation judges, seal the test rows
Test transfer	Do not consult the ratings again	Compute the frozen screen on the held-out judges; apply the pre-registered gates once
Screen a new judge	None	Run the frozen probes; inspect the index and the failure report

Table 1: **Screening a new judge costs inference, not ratings; the transfer test is a one-shot evaluation on judges the screen was not selected on.** The split is over judge models and harnesses, not over items, because the question is whether the screen carries over to a new judge.

repeated calls, averaged over the repeated cells. For a deletion ladder, level 0 is the gold as its own candidate and level ℓ removes a cumulative fraction ℓ/L of its theorem statements, last theorem first, with $L \leq 4$. The ladder quality is $(1-\rho)/2$ for the tie-aware Spearman correlation ρ between level and median score; a constant ladder counts 0. $C_{\mathrm{ms,gold}}$ is the mean ladder quality over the 24 gold ladders, so a score that falls monotonically with every deletion gives 1 and a constant or rising score gives 0. A component needs at least 80% of its valid probes scored; otherwise it is missing rather than zero.

The candidate family and the selection rule. Twenty-two indices were pre-registered: the two indices of our earlier work, nine fixed aggregates of component subsets (geometric means, an arithmetic mean, and a minimum), and the ten single components. The rule ranks every candidate by its Spearman association with the criterion over the 18 train configurations, breaking ties toward fewer components, keeps the top three, and selects the finalist with the highest validation association. The test rows were sealed in a private file, and the selection was committed before any test row was read.

Pre-registered gates. The selected index passes if its test Spearman association is at least 0.45 and its one-sided permutation p for a positive association is below 0.05 (10,000 permutations). Uncertainty is a 95% percentile bootstrap over configurations (10,000 draws). The outcome is inconclusive, never resolved by changing a threshold, if the criterion is not computable for more than five of the 18 test configurations or if the interval spans more than 1.2. We also report whether the train, validation, and test associations lie within 0.30 of each other, and the two earlier indices on every split regardless of selection. The permutation test rejects the absence of a positive association; it does not test that the population correlation exceeds 0.45, so we report the interval's lower bound beside the verdict.

The two earlier indices. $\mathrm{TI}_{\mathrm{var}}$ is the geometric mean of the mean bug-ladder quality and two variance penalties, min–max normalized here over the train configurations (Equation 1, Appendix B). PTI_{v0} is the frozen geometric mean of the identity, bug-ladder, deletion-ladder, and alpha-renaming properties with exact-zero geometric means and strict coverage, so it is undefined whenever any required probe is unscored and zero whenever any property is zero. Both enter the family as frozen candidates and are reported on every split.

5 Experiments

5.1 Setup

Judges. Fifty-six LLM configurations combine six models, nine prompts, and two harnesses. The train judges are GPT-6 Sol (high reasoning) and Claude Sonnet 5 (medium effort); the validation judges are GPT-6 Luna (high), Gemini 3.1 Pro, and GPT-6 Luna inside the Terminus2 agent harness; the test judges are Claude Opus 5.5 (maximum effort), Qwen3.6-27B, and Terminus2 with Claude

Sonnet 5 and with Qwen3.6-27B. Five fuller-context prompts come from VERIBENCH's judge: the deployed coverage rubric (D0), an earlier checklist with a precondition clause (R4), the raters' own instructions verbatim (H0), those instructions with four rater conventions (H1), and with rated demonstrations added (H2). Four legacy prompts come from our earlier version and show the judge only the gold's last theorem block: a baseline (P1), a checklist (P4), a strict variant (P8), and the deliberately lexical `BAD_STRINGY` (LEX). Claude Opus 5.5 ran five prompts and each agent judge two, so the split holds 18 train, 20 validation, and 18 test configurations; four deterministic controls join the train split for reporting only. Every call went through the Vals-hosted gateway, with Vals-reported charges of $574.

Items and criterion. The items are 75 candidates from 25 VERIBENCH tasks, three agent generations per task, with the references current at the time of rating. The criterion items are the 45 candidates of the 15 tasks in VERIBENCH's validation and test task split, and the target is the mean of three expert coverage ratings on a 0–5 scale, divided by five. A configuration's criterion is the Spearman correlation between its median of three repeated scores and the target, computed only when at least 90% of the 45 items have a score. The mean leave-one-expert-out Spearman correlation among the raters on these items is 0.747. Across 47 LLM configurations, the criterion computed on the 15 validation items and on the 30 test items rank the configurations alike (Spearman 0.928), so the item split is not what separates judges.

Probes. Twenty-four of the 25 gold references elaborate under Lean 4.22 with Mathlib, giving 24 identity cells, 25 empty-candidate cells, 76 valid deletion levels on 24 ladders, and 37 valid bug levels. Only three of the 75 rated candidates elaborate under this toolchain, because of retired imports, Markdown fences, and type errors, so the candidate-side probes cover three candidates and the reordering probe none. The campaign declared 28,591 cells over the 56 configurations: 28,345 were measured, 504 were settled by the deployed rubric's deterministic rules, and 327 are missing by class (235 unparsable verdicts, 81 extractor refusals, 7 model errors, 4 infrastructure failures) and stay missing in every denominator.

5.2 WHAT TRANSFERS TO HELD-OUT JUDGES

Table 2 and Figure 2 give the result. Gold deletion monotonicity, $C_{\mathrm{ms,gold}}$, won the train ranking (Spearman 0.761 over 18 configurations) and the validation check (0.762 over 20), ahead of the identity component and a two-component geometric mean. On the held-out judges its association with the criterion is 0.558, with a 95% configuration-bootstrap interval of $[0.078, 0.762]$ and a one-sided permutation p of 0.0127 over the 16 of 18 test configurations for which it is defined. Both pre-registered gates pass, the criterion was computable for all 18 test configurations, and the train–test gap of 0.203 is within the 0.30 tolerance. The two missing configurations are Qwen3.6-27B with the H2 and R4 prompts, whose deletion ladders fell below the 80% coverage rule after unparsable verdicts. The interval's lower bound is 0.078, and the permutation test does not establish that the population association exceeds 0.45.

Scope limit: the prompt groups. The legacy prompts show the judge only the gold's last theorem block, so deleting statements elsewhere in the reference is invisible to them beyond the first level; this limitation was recorded before any call. The legacy configurations therefore sit at low $C_{\mathrm{ms,gold}}$ and low agreement, and the fuller-context configurations at high values of both (Figure 2). Excluding the deliberately lexical prompt leaves 14 test configurations with an association of 0.453 $[-0.177, 0.825]$ and $p = 0.0551$, below the registered gates. Within the ten fuller-context test configurations the association is -0.371 $[-0.787, 0.304]$, $p = 0.8615$. This subset mixes rubrics and models and is not a controlled comparison, and the human-mirror rubric alone has only three test configurations, below the protocol's minimum of eight. The registered result therefore shows that the screen separates judges that see the reference from judges that do not; it does not show that the screen ranks judges within the group one would deploy.

What does not transfer. $\mathrm{TI}_{\mathrm{var}}$, the scalar our earlier version proposed, has no positive association on any split and -0.332 $[-0.636, 0.135]$ on the 18 test configurations. PTI_{v0} is undefined for the validation judges and for nine test configurations, because its exact-zero geometric mean needs every ladder and the three alpha-renaming probes, and it is zero for eight of the nine test configurations

Index	Judge split	n	Spearman [95% interval]	Pearson	Kendall τ_b	Permutation p
$C_{\mathrm{ms,gold}}$ (selected)	train	18	0.761	0.643	0.569	—
	validation	20	0.762	0.759	0.537	—
	test	16 of 18	**0.558** [0.078, 0.762]	0.643	0.360	**0.0127**
test without the lexical prompt	test	14	0.453 [−0.177, 0.825]	0.824	0.331	0.0551
test, fuller-context prompts only	test	10	−0.371 [−0.787, 0.304]	−0.319	−0.225	0.8615
$\mathrm{TI}_{\mathrm{var}}$ (earlier scalar)	train	18	−0.010	0.128	−0.013	—
	validation	20	−0.177	−0.042	−0.132	—
	test	18	−0.332 [−0.636, 0.135]	−0.194	−0.172	0.9135
PTI_{v0} (earlier four-property index)	train	15	0.433	0.307	0.365	—
	validation	0 of 18	not computable	—	—	—
	test	9 of 18	−0.274 [−0.753, 0.111]	−0.075	−0.236	0.7793

Table 2: **Gold deletion monotonicity, selected on the train and validation judges, predicts the held-out judges' agreement with the experts under the pre-registered gates, but the association depends on the prompt group; the two earlier indices do not transfer.** Each row is a judge-level Spearman association between an index and the criterion over judge configurations; n counts configurations with both values. Intervals are 95% percentile bootstraps over configurations and p is the one-sided positive-association permutation probability (10,000 draws, seed 39), computed for the test split only; the two prompt-group rows are post hoc sensitivity analyses on the same test rows. PTI_{v0} is undefined wherever any required probe is unscored, and eight of its nine test values are exactly zero.

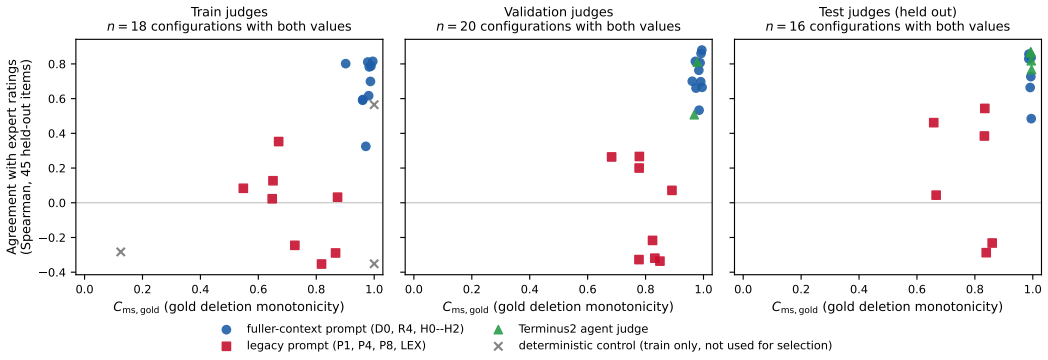


Figure 2: **The association between gold deletion monotonicity and agreement with the experts is carried largely by the gap between the legacy prompts and the fuller-context prompts.** Each marker is one judge configuration; the horizontal axis is $C_{\mathrm{ms,gold}}$ and the vertical axis its Spearman agreement with the expert ratings on the 45 criterion items. The fuller-context configurations cluster near $C_{\mathrm{ms,gold}} = 1$, where the component no longer separates them; the deterministic controls appear on the train panel for reference and were not used for selection. Drawn from the committed unit files of the transfer study.

where it is defined, giving -0.274 $[-0.753, 0.111]$. These rows are the pre-registered secondary report; they share the test judges with the selected index but not its 16-configuration support, so the comparison is descriptive rather than a paired test. Two further descriptive facts: the identity component, a finalist that was not selected, reaches 0.697 on all 18 test configurations, and the four agent-judge configurations occupy the top of both axes (criterion 0.768–0.870, $C_{\mathrm{ms,gold}}$ 0.992–0.996).

5.3 Does the screen choose better judges than a fixed prompt?

A screen is useful in practice if choosing a judge by it yields a judge that agrees with the raters at least as well as a simple alternative. We compared four selectors on the 12 fuller-context test configurations, using only cached scores: choose the configuration with the highest $C_{\mathrm{ms,gold}}$; choose uniformly among the four configurations that use the human-mirror rubric H2, which VeriBench transferred from its train judges; choose uniformly at random; and choose by the identity component. Agreement is evaluated on the 42 rated files from 15 tasks that every configuration scored, with paired task-cluster

Selector (12 fuller-context test configurations)	Expected Spearman	95% interval	Regret vs. best	Mean bias (judge − human)
Proxy: highest $C_{\mathrm{ms,gold}}$	0.768	$[0.401, 0.930]$	0.104 $[0.008, 0.379]$	-0.010
Fixed human-mirror rubric H2	0.853	$[0.645, 0.931]$	0.018 $[0.009, 0.116]$	$+0.120$
Random configuration	0.784	$[0.528, 0.892]$	0.087 $[0.047, 0.238]$	$+0.097$
Identity component	0.784	$[0.528, 0.892]$	0.087 $[0.047, 0.238]$	$+0.097$
Proxy − H2	-0.086	$[-0.308, 0.058]$		
Proxy − random	-0.017	$[-0.162, 0.080]$		

Table 3: **Choosing a judge by the transferred screen does not beat a fixed human-mirror prompt on cached scores.** Expected Spearman agreement with the three-expert mean over the 42 shared rated files from 15 tasks; intervals are paired task-cluster percentile bootstraps (10,000 draws) conditional on the fixed pool, with no hypothesis test (p-val not applicable). The identity component ties all twelve configurations, so it coincides with random choice. Mean bias is the selected judge's mean score minus the human mean (0.502) on the same files, positive meaning lenient; mean absolute errors are 0.154 (proxy) and 0.173 (H2).

bootstraps of 10,000 draws conditional on this fixed pool. The proxy selects the Terminus2 agent judge with the deployed rubric, with expected Spearman agreement 0.768 $[0.401, 0.930]$; the fixed H2 rubric gives 0.853 $[0.645, 0.931]$ and random choice 0.784 $[0.528, 0.892]$ (Table 3). The paired difference between the proxy and the fixed rubric is -0.086 $[-0.308, 0.058]$, and between the proxy and random choice -0.017 $[-0.162, 0.080]$; no hypothesis test was run. The proxy's regret against the best configuration in the pool is 0.104 $[0.008, 0.379]$, the fixed rubric's 0.018 $[0.009, 0.116]$. Every reference-quality view (excluding placeholder references, ambiguous references, both, or restricting to the test tasks) keeps the proxy at or below the fixed rubric, by 0.050 to 0.177. Re-targeting the criterion to all five current raters on the test tasks alone gives proxy 0.815 $[0.411, 0.950]$ against 0.826 $[0.494, 0.936]$ for the fixed rubric, a paired difference of -0.012 $[-0.198, 0.186]$ over 29 of 30 outputs from 10 tasks. Selection utility is therefore not demonstrated; it is also not disproved, and the intervals are too wide to call the two selectors equivalent.

5.4 A SEMANTIC SCREEN: VACUITY, WEAKENING, AND OVER-CONSTRAINT

The deletion ladders remove whole statements. To test whether judges notice specifications that look right but guarantee less, we built 194 Lean-validated probes from the same 24 references: 24 identity anchors, 38 vacuity probes that replace theorem statements by `True`, 57 cumulative weakenings (a dropped conjunct, a relaxed equality, a disjunctive escape) each with a `sorry`-free Lean proof that the reference statement implies its weakened counterpart, 37 over-constraint probes that add a hypothesis proved unsatisfiable, and 38 formatting-only edits. All 56 configurations scored every probe three times (32,592 calls, 99.35% measured, $656.15). Directional accuracy is the share of a family's probes scored strictly below the same task's identity probe. The locked rule requires directional accuracy of at least 0.8 on vacuity, weakening, and over-constraint and a formatting gap of at most 0.05. Vacuity is caught almost universally (mean directional accuracy 0.996). Weakening separates judges: under the fuller-context rubrics Claude Sonnet 5 reaches 0.95 and Qwen3.6-27B 0.91, Claude Opus 5.5 0.87, Gemini 3.1 Pro 0.68, GPT-6 Luna 0.42, and GPT-6 Sol 0.38, and the legacy prompts, which cannot see most weakened statements, 0.04–0.14. Fifteen of the 56 configurations pass: Claude Sonnet 5 and Qwen3.6-27B on all five fuller-context rubrics, Claude Opus 5.5 on two, Gemini 3.1 Pro on one, and both Terminus2 judges with the H2 rubric. The screen is necessary rather than sufficient: a character edit-distance baseline passes it because every edit lowers its score, which the formatting false-alarm rate exposes (its discrimination, semantic accuracy minus false alarms, is 0.00). Thirty-one of the 57 weakening probes leave the file-level guarantee unchanged because another theorem of the same reference restates the dropped property; on the 25 strictly weaker probes GPT-6 Sol with H2 rises from 0.39 to 0.84 and GPT-6 Luna with H2 from 0.44 to 0.92, so part of the miss rate is judges declining to penalize redundancy. Weakening directional accuracy also tracks the criterion: Spearman 0.73 $[0.55, 0.86]$ over the 38 train and validation configurations ($p < 0.001$) and 0.72 $[0.31, 0.92]$ over the 18 test configurations ($p = 0.001$), the latter computed after the test receipt existed. Combining a semantic score with $C_{\mathrm{ms,gold}}$ changes the train and validation association from 0.78 to 0.72 ($\Delta = -0.06$ $[-0.16, 0.04]$) and the test association from 0.56 to 0.67 ($\Delta = +0.11$ $[-0.13, 0.39]$), so no incremental benefit is established. These unit bootstraps resample configurations that share six judge models and understate the uncertainty.

Attacked judge (Claude Sonnet 5)	Gaming gap [95% interval]	Attacked gain	Reference gain	Flagged, edited cells
LEX (lexical)	-0.162 $[-0.325, -0.033]$	$+0.013$	$+0.176$	7 of 8
D0 (deployed rubric)	-0.044 $[-0.186, +0.048]$	$+0.311$	$+0.356$	13 of 15
H0 (raters' instructions)	-0.027 $[-0.057, -0.002]$	$+0.236$	$+0.262$	7 of 7
H2 (human mirror)	$+0.033$ $[-0.103, +0.142]$	$+0.378$	$+0.344$	10 of 10
P1 (legacy baseline)	-0.136 $[-0.281, +0.031]$	$+0.164$	$+0.300$	11 of 11
P8 (legacy strict)	-0.088 $[-0.213, +0.067]$	$+0.086$	$+0.173$	11 of 11

Table 4: **No attacked configuration gained more than the model reference credited, and nearly every edited candidate carries a flag that the reference credited too.** Fifteen cells per judge; gains are fresh-repeat means on the $[0, 1]$ scale; intervals are task-clustered percentile bootstraps over the six tasks (10,000 resamples, seed 43), with no hypothesis test. Flag counts are edited cells whose final candidate adds a vacuous, ground, unrelated, duplicated, or inflated statement, among the cells the attacker changed.

5.5 A SCORE-OPTIMIZING ATTACKER

We also tested resistance to optimization directly. GPT-6 Luna (high reasoning) edited 15 low-coverage rated candidates (expert mean at most 0.6, six tasks) for eight Lean-checked rounds to raise one judge's score, seeing only that score, the task's Python source, and its own history, never the gold, the rubric, or a rating. It attacked six Claude Sonnet 5 configurations (LEX, D0, H0, H2, P1, P8): 90 cells and 720 attempts, of which 634 elaborated, 356 only after failing proofs were replaced by `sorry`; 119 edits were accepted and 62 of the 90 final candidates differ from their start. Each start and final candidate was then rescored three times by the attacked judge and by a reference of the H2 rubric on Claude Opus 5.5 and on GPT-6 Sol. The gaming gap, the attacked judge's gain minus the reference's gain, ranges from -0.162 $[-0.325, -0.033]$ for the lexical prompt to $+0.033$ $[-0.103, +0.142]$ for H2, with no task-bootstrap interval wholly above zero (Table 4). This is no evidence that the edits were sound. The edited files grew from 0.82 to 12.37 theorem statements on average, and 59 of the 62 edited candidates carry at least one deterministic flag, most often a unit test stated as a theorem (50 of 62) or a statement that never mentions the implementation (45 of 62); the reference credited those files as well. The defensible reading is that no attacked judge shows a statistically resolved excess over the model reference, not that any judge resisted. Across the six attacked configurations the label-free index did not predict the gap: $C_{\mathrm{ms,gold}}$ has Spearman $+0.600$ with the gap (exact one-sided $p = 0.912$ for the resistance direction), $\mathrm{TI}_{\mathrm{var}}$ -0.086 ($p = 0.460$), and PTI_{v0} is defined for only three of the six.

6 LIMITATIONS AND A DECISIVE TEST

What the transfer result shows and what it does not. The registered gates test whether the association on the held-out judges is positive; they do not test whether the population association exceeds 0.45, and the interval's lower bound is 0.078. The association is carried largely by the difference between prompts that see the whole reference and prompts that see its last theorem block, and it reverses within the ten fuller-context test configurations. The screen therefore separates judge groups one can already tell apart; within the group one would deploy, the evidence does not show that it ranks judges. The configurations inside each split share judge models and prompt families, so the configuration bootstrap understates uncertainty, and the test judges were two single-call models and two agent harnesses rather than an independent sample of judges.

No demonstrated selection advantage. Choosing by the screen did not beat a fixed human-mirror prompt on cached scores (paired difference -0.086 $[-0.308, 0.058]$), and the five-rater sensitivity leaves the comparison unresolved (-0.012 $[-0.198, 0.186]$). Both comparisons reuse the criterion items, condition on a fixed pool of twelve configurations, and run no hypothesis test; they cannot show that the screen is useless, only that its usefulness is not established.

Coverage of the probes. Only three of the 75 rated candidates elaborate under the pinned toolchain, so the candidate-side invariance components (alpha-renaming, proof-to-`sorry`, reordering, spam, candidate deletion) rest on three candidates or none, and every pre-registered aggregate that contains reordering was undefined. The family effectively reduced to the gold-side components and the

single components, and we claim nothing about rewrite invariance. Elaboration failures reflect retired imports and Markdown fences in the stored candidates, not necessarily invalidity under the environments in which they were generated. The gold deletion ladders remove theorem statements and unit tests alike (Appendix G), and the legacy prompts can see only the first deletion level.

The criterion. The criterion is one human panel: 45 candidates from 15 tasks, 18 with two expert ratings and 27 with three, and a leave-one-expert-out ceiling of 0.747. Nine of the 45 candidates come from three tasks whose reference was a placeholder at rating time, and nine from three tasks whose reference statements changed within the rating window. The reference-quality views of the selection-utility study did not change its reading, but a criterion restricted to ordinary references has not been used for the transfer test itself. Prompt families and rated items recur across the judge splits, so the result supports transfer to new judge configurations on this panel, not to new candidate generators, domains, or independent panels.

The characterizations' own limits. The semantic screen is necessary, not sufficient: a change detector passes it, about half of its weakening probes are redundant at file level, and two references contain mutually inconsistent `sorry`-proved theorems. The gaming study used one attacked model, six prompts, 15 items from six tasks, and a model ensemble as its reference, so a near-zero gap can hide a blind spot the reference shares, and its six-point association can detect only very strong relations. The edited candidates have no human ratings; an expert re-rating is the only way to settle whether their gains were genuine. Presentation, including redundant theorems and unnecessary abstraction, is unassessed, and all candidates and probes come from VERIBENCH in Lean 4.

A decisive test. A decisive test of the screen as a selector would freeze the probes, the rubric, the extraction, and a selection rule, then have independent raters rate new candidate–gold pairs under the frozen rubric for configurations fixed before any rating is examined. It would report the selection loss against the best available judge and a false-acceptance rate, with a sample size chosen from clustered precision rather than an arbitrary judge count. Human adjudication of weakened and attacker-edited files, which both model judges and the model reference credited, would test the shared blind spot that no model-only study can.

7 CONCLUSION

Agents now write the trust artifacts for their own code, and a language-model judge decides whether those artifacts say the right thing. We asked whether label-free behavioral properties can take over part of the human validation of that judge, and we tested the answer on judges the properties were not selected on. One property transfers: the fall in score as theorem statements are deleted from the reference predicts held-out judges' agreement with expert ratings under our pre-registered gates, with a wide interval and an association carried by prompt groups rather than by differences among the judges one would deploy. The two indices our earlier work proposed do not transfer, choosing a judge by the transferred screen does not beat a fixed human-mirror prompt, a semantic screen passes 15 of 56 configurations, and a score-optimizing attacker inflates the attacked judges and the validated reference alike. Label-free screens can rule judges out and expose failures that human agreement hides; choosing among the judges one would deploy still needs a human panel, and the decisive test of any replacement is a frozen screen scored against new ratings.

Reproducibility Statement

Every number in Sections 5.2–5.5 is read from committed receipts of the four studies behind them: the transfer study's locked protocol, probe manifest, per-configuration harvests, sealed selection and test receipts; the selection-utility study's cached-score analysis; the semantic screen's Lean-validated probe manifest, per-configuration tables, and association receipt; and the gaming study's trajectories and analysis receipt. The train, validation, and test rows of Table 2 and Figure 2 are recomputed from the committed unit files without model calls, as are the two prompt-group rows, the selection-utility comparison, and the gaming association. Every judge call went through one hosted gateway with recorded model identifiers, prompt hashes, and per-cell status, and missing cells stay missing in every denominator. Appendix A names the judge models and prompt families; Appendix B specifies the components and indices, including numerical edge cases. The earlier 72-pair validation set and its frozen score vectors reproduce the appendix tables exactly (Appendix C). The expert ratings that define the criterion were collected by VeriBench under its own rating protocol and are reused here unchanged.

Ethics Statement

The criterion ratings were supplied by expert raters recruited for VeriBench; the earlier validation set, now in the appendix, was rated by an author. The study evaluates generated Lean code and formal statements, without personal data in the judged artifacts. It makes no deployment claim and does not establish that screened judges can replace expert review in consequential settings.

Artificial Intelligence Use Statement

LLMs generated the candidate Lean artifacts, served as the judges under study, acted as the score-optimizing attacker, and formed the reference ensemble of the gaming study. Coding-assistant subagents proposed the weakening edits of the semantic screen, which were admitted only through the Lean checks, and helped implement experiments and analysis scripts, check and interpret results, and prepare figures. They also assisted with methodological feedback, literature retrieval, manuscript drafting, and editing under author supervision. The human ratings used throughout were supplied by human raters, not generated by these tools. The authors are responsible for the manuscript's claims, citations, and interpretation of the recorded results.

References

Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785.

Anna Bavaresco, Raffaella Bernardi, Leonardo Bertolazzi, Desmond Elliott, Raquel Fernández, Albert Gatt, Esam Ghaleb, Mario Giulianelli, Michael Hanna, Alexander Koller, André F. T. Martins, Philipp Mondorf, Vera Neplenbroek, Sandro Pezzelle, Barbara Plank, David Schlangen, Alessandro Suglia, Aditya K. Surikuchi, Ece Takmaz, and Alberto Testoni. Llms instead of human judges? a large scale empirical study across 20 nlp evaluation tasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 238–255. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-short.20. URL https://aclanthology.org/2025.acl-short.20/.

Guoxin Chen, Jing Wu, Xinjie Chen, Wayne Xin Zhao, Ruihua Song, Chengxi Li, Kai Fan, Dayiheng Liu, and Minpeng Liao. ReForm: Reflective autoformalization with prospective bounded sequence optimization. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/419bd6a89ab56e9c09ededb560b06192-Abstract-Conference.html.

T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, The Hong

Kong University of Science and Technology, 1998. URL https://arxiv.org/abs/2002.12543. arXiv:2002.12543.

Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015. doi: 10.1126/science.aaa9375. URL https://doi.org/10.1126/science.aaa9375.

Sabrina Haque, Sarvesh Ingale, and Christoph Csallner. Do autonomous agents contribute test code? a study of tests in agentic pull requests, 2026. URL https://arxiv.org/abs/2601.03556.

Shuvendu K. Lahiri. Evaluating LLM-driven user-intent formalization for verification-aware languages. In *Proceedings of the 24th Conference on Formal Methods in Computer-Aided Design (FMCAD)*, pp. 142–147. TU Wien Academic Press, 2024. doi: 10.34727/2024/isbn.978-3-85448-065-5_19. URL https://arxiv.org/abs/2406.09757.

Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover: A frontier model for open-source automated theorem proving, 2025. URL https://arxiv.org/abs/2502.07640.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2511–2522. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.153. URL https://arxiv.org/abs/2303.16634.

Jianqiao Lu, Yingjia Wan, Yinya Huang, Jing Xiong, Zhengying Liu, and Zhijiang Guo. FormalAlign: Automated alignment evaluation for autoformalization. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/fceedf8c9c0ff51f41b9fe0294ef0070-Abstract-Conference.html.

Brando Miranda, Srivatsava Daruru, Ethan S Hersch, Zhanke Zhou, Allen Nie, Daneshvar Amrollahi, Leni Aniva, Iddah Mlauzi, Kirill Acharya, Elyas Obbad, Dilara Soylu, Weston Kirk, Zixiao Jolene Wang, Kai Fronsdal, Ying Li, Donald Poindexter Jr, Rakshit Kaushik, Shurui Liu, Yegor Denisov-Blanch, Steven Dillmann, Simon Obstbaum, Santiago Cuellar, John Sarracino, Rylan Schaeffer, Mo Tiwari, Donghyun Lee, Bo Han, and Sanmi Koyejo. Veribench: An end-to-end formal verification benchmark for AI coding agents in lean 4. In *3rd AI for Math Workshop: Toward Self-Evolving Scientific Agents*, 2026. URL https://openreview.net/forum?id=lkL0qnUv3p.

Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-bench: Testing and validating real-world bug-fixes with code agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2406.12952.

Auguste Poiroux, Gail Weiss, Viktor Kunčak, and Antoine Bosselut. Reliable evaluation and benchmarks for statement autoformalization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 17947–17969. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.907. URL https://aclanthology.org/2025.emnlp-main.907/.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442. URL https://aclanthology.org/2020.acl-main.442/.

Zachary Robertson and Sanmi Koyejo. Let's measure information step-by-step: AI-based evaluation beyond vibes, 2025. URL https://arxiv.org/abs/2508.05469.

Kayla Schroeder and Zach Wood-Doughty. Can you trust llm judgments? reliability of llm-as-a-judge, 2024. URL https://arxiv.org/abs/2412.12509.

Weixi Tong and Tianyi Zhang. Codejudge: Evaluating code generation with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 20032–20051. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.1118. URL https://arxiv.org/abs/2410.02184.

Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, Jianqiao Lu, Hugues de Saxcé, Bolton Bailey, Chendong Song, Chenjun Xiao, Dehao Zhang, Ebony Zhang, Frederick Pu, Han Zhu, Jiawei Liu, Jonas Bayer, Julien Michel, Longhui Yu, Léo Dreyfus-Schmidt, Lewis Tunstall, Luigi Pagani, Moreira Machado, Pauline Bourigault, Ran Wang, Stanislas Polu, Thibaut Barroyer, Wen-Ding Li, Yazhe Niu, Yann Fleureau, Yangyang Hu, Zhouliang Yu, Zihan Wang, Zhilin Yang, Zhengying Liu, and Jia Li. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning, 2025. URL https://arxiv.org/abs/2504.11354.

Kaiyu Yang, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin Lauter, Swarat Chaudhuri, and Dawn Song. Formal mathematical reasoning: A new frontier in AI, 2024. URL https://arxiv.org/abs/2412.16075.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, pp. 46595–46623. Curran Associates, Inc., 2023. doi: 10.52202/075280-2020. URL https://arxiv.org/abs/2306.05685.

Terry Yue Zhuo. ICE-score: Instructing large language models to evaluate code. In Yvette Graham and Matthew Purver (eds.), *Findings of the Association for Computational Linguistics: EACL 2024*, pp. 2232–2242, St. Julian's, Malta, March 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-eacl.148. URL https://aclanthology.org/2024.findings-eacl.148/.

A JUDGE PROMPTS AND SCORING CONFIGURATION

Judges and prompts of the transfer study (Section 5.1). The six judge models were served through one hosted gateway under the identifiers `openai/gpt-6-sol-high` (reasoning effort high), `anthropic/claude-sonnet-5-medium` (compute effort medium), `openai/gpt-6-luna-high` (high), `google/gemini-3.1-pro-preview`, `anthropic/claude-opus-5-5-no-fallback` (compute effort maximum), and `alibaba/qwen3.6-27b`. The agent judges ran the same models inside the Terminus2 harness (at most 20 turns, reasoning effort high where exposed) and wrote their verdict to a file. The five fuller-context prompts are VERIBENCH's frozen rubrics, imported byte-for-byte: `D0_deployed_tc_v1` (integer 0–10, divided by ten), `R4_exp75_human_precondition`, `H0_human_mirror`, `H1_human_mirror_rater_conventions`, and `H2_human_mirror_conventions_demos` (integer 0–5, divided by five); each shows the judge the complete reference, the task's Python source, and the full candidate. The four legacy prompts are the `P1_BASELINE`, `P4_CHECKLIST`, `P8_STRICT_B`, and `BAD_STRINGY` prompts listed below, verbatim, with the reference field holding the gold's last theorem block as in our earlier version. Scores were normalized to $[0, 1]$ at parse time, and the expert target is the 0–5 mean divided by five, so no further scaling is applied.

Prompts of the earlier 72-pair validation set (Appendix C). The primary family model identifiers are `gpt-5.3-codex`, `claude-opus-4-7`, and `Qwen/Qwen3-Coder-30B-A3B-Instruct`. Each family panel contains the three semantic prompts below and `BAD_STRINGY`, plus `gpt-4o-mini` with the checklist and strict prompts. The other two configurations are the deterministic edit-distance control `editdist_e1` and seeded control `random_r1`. All model prompts request scores in $[0, 1]$, not integers on a 0–10 scale. The human ratings use 0–5; correlations compare their ordering with judge scores without equating the scales.

The reference field retains its historical label "REFERENCE IMPLEMENTATION," but the code fills it with the gold's final theorem block. The candidate field contains the full candidate file for human-agreement scoring; property probes operate on the extracted theorem text. If no theorem block is found, the extractor uses the full file. The Codex panel aggregates three recorded calls for model configurations and nine for controls by the median. The Opus and Qwen panels aggregate three calls for every configuration by the median. These counts come from the saved repeat arrays; a summary-level repeat setting alone does not describe reused cached rows. The random control is deterministic for its input and seed, so repeated identical scores do not imply a reliable semantic judge. Prompt labels P1, P4, and P8 are historical identifiers and do not denote the property numbers P1–P4.

P1_BASELINE

```
You are an expert in Lean 4 formal verification.
Score how well the GENERATED THEOREM captures the correctness properties
of the REFERENCE IMPLEMENTATION. Score from 0.0 to 1.0.

- 1.0: Semantically equivalent to what the reference guarantees
- 0.7-0.9: Captures most properties, misses minor ones
- 0.4-0.6: Captures some properties, has meaningful gaps
- 0.1-0.3: Technically valid Lean but misses core properties
- 0.0: Trivially true (e.g., `True`) or incorrect

REFERENCE IMPLEMENTATION:
{reference_impl}

GENERATED THEOREM:
{generated_theorem}

Respond ONLY with valid JSON, no markdown:
{"score": <float 0.0-1.0>,
 "reasoning": "<one sentence max>"}
```

P4_CHECKLIST

```
Evaluate this in order:
1) What properties does REFERENCE IMPLEMENTATION guarantee?
2) Which are preserved by GENERATED THEOREM?
3) Are there missing core guarantees or trivialization?
```

```
4) Map to score in [0.0, 1.0]:
   1.0 equivalent; 0.7-0.9 minor gaps;
   0.4-0.6 meaningful gaps; 0.1-0.3 weak;
   0.0 trivial/incorrect.

REFERENCE IMPLEMENTATION:
{reference_impl}

GENERATED THEOREM:
{generated_theorem}

Return JSON only:
{"score": <float 0.0-1.0>,
 "reasoning": "<one sentence>"}
```

P8_STRICT_B

```
Evaluate conservatively. If uncertain, choose the lower score.
Use [0.0,1.0]:
1.0 equivalent semantics
0.7-0.9 mostly right, minor misses
0.4-0.6 substantial omissions
0.1-0.3 weak relation
0.0 trivial/incorrect

REFERENCE IMPLEMENTATION:
{reference_impl}

GENERATED THEOREM:
{generated_theorem}

Return valid JSON only:
{"score": <float>,
 "reasoning": "<one sentence>"}
```

BAD_STRINGY negative control

```
You are a text similarity grader.
Compare generated theorem text to the reference implementation text.
Do NOT evaluate semantic correctness or theorem meaning.
Score ONLY based on superficial lexical/format overlap:
- high score if tokens/phrases look similar
- low score if wording/structure differs
Reference implementation:
{reference_impl}

Generated theorem:
{generated_theorem}

Return ONLY a JSON object:
{"score": <float 0.0 to 1.0>,
 "reasoning": "<short reason>"}
```

B Index Definitions and Implementation

Components of the transfer study. Let $m(k, \kappa, \ell)$ be the median over repeated calls of a configuration's score on cell (k, κ, ℓ), where k is the probe kind, κ the gold or candidate, and ℓ the ladder level. For a gold ladder of kind $k \in \{\text{deletion}, \text{bug}\}$ on gold κ with valid levels $1, \ldots, L$, the level sequence is $(0, 1, \ldots, L)$ and the score sequence is $(m(\text{ident}, \kappa, 0),\ m(k, \kappa, 1), \ldots, m(k, \kappa, L))$; a ladder with any unscored level is dropped. Its quality is $q = (1 - \rho)/2$ with ρ the tie-aware Spearman correlation between the two sequences, and $q = 0$ when every score is identical. $C_{\text{ms,gold}}$ and $C_{\text{bm,gold}}$ are the means of q over the gold deletion and bug ladders, and $C_{\text{ms,cand}}$ applies the same rule to candidate deletion ladders anchored at the criterion cell. Identity is the mean of $m(\text{ident}, \kappa, 0)$ over golds; blank is the mean of $1 - m(\text{blank}, \kappa, 0)$. Alpha-renaming, reordering, and proof-to-`sorry` invariance are means of $1 - |m(\text{crit}, x, 0) - m(k, x, 1)|$ over candidates x; spam non-increase is the mean of $1 - \max(0,\ m(\text{spam}, x, 1) - m(\text{crit}, x, 0))$. Stability is the mean of $1 - \min(1,\ 2\,\sigma)$ over the criterion, identity, and bug cells with at least two repeats, where σ is the population standard deviation across repeats. A component is reported only when at least 80% of its structurally valid probes are scored; otherwise it is missing, and so is every index that contains it. The pre-registered aggregates take geometric means, an arithmetic mean, or a minimum of fixed component subsets without fitted weights.

The frozen four-property index. PTI_{v0} is the geometric mean of four properties: the geometric mean of the identity scores over golds, the geometric means of the bug-ladder and deletion-ladder qualities, and the geometric mean of the alpha-renaming invariances over candidates. Its geometric means are exact-zero (any zero factor gives zero, with no floor), and each property requires every structurally valid probe to be scored, so the index is undefined whenever a required score is missing. Single-repeat ladder levels contribute their quality unchanged.

Fixed equal-weight index of the earlier version. The scalar our earlier version proposed is computed from bug-ladder scores only:

$$\mathrm{TI}_{\mathrm{var}} = \exp\Big[\tfrac{1}{3}\log C_f + \tfrac{1}{3}\log(1-\widetilde{V}_c) + \tfrac{1}{3}\log(1-\widetilde{V}_s)\Big]. \tag{1}$$

In the transfer study the tildes denote min–max normalization over the 18 train configurations, so the index is panel-independent for the validation and test judges. For each file f and bug level ℓ, let $s_{f\ell}$ be the median recorded repeat score and $v_{f\ell}$ its population variance. Let ρ_f be Spearman's correlation between level and $s_{f\ell}$, and set $c_f = \mathrm{clip}((1-\rho_f)/2, 0, 1)$. Writing $\mathcal{F}$ for the included ladders, the archived implementation uses

$$C_f = \frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}} c_f, \qquad V_c = \mathrm{Var}_{f\in\mathcal{F}}(c_f), \qquad V_s = \frac{\sum_{f\in\mathcal{F}}\sum_{\ell} v_{f\ell}}{\sum_{f\in\mathcal{F}} L_f},$$

where L_f is the number of levels in ladder f. It skips ladders with fewer than three levels or an undefined Spearman correlation, including constant-score ladders. All primary committed vectors report 24 included ladders per configuration. If none remain, the code substitutes $C_f = 0$, $V_c = 1$, and $V_s = 1$. It min–max normalizes each variance across the current family panel; a constant variance column maps to zero. Equation 1 then applies weights $1/3$ to C_f, $1-\widetilde{V}_c$, and $1-\widetilde{V}_s$, after clipping each to $[10^{-6}, 1]$. Thus an extreme variance does not produce an exact zero index, and adding a judge can change other judges' normalized scores. The analysis preserves these choices rather than changing the frozen index after observing results.

Arithmetic comparisons. The recorded $v1$ and $v2$ are

$$v1 = 0.30\,\mathrm{ident} + 0.30\,\mathrm{mono} + 0.25\,\mathrm{weak} + 0.15\,\mathrm{stab},$$

$$v2 = 0.20\,\mathrm{ident} + 0.20\,\mathrm{mono} + 0.15\,\mathrm{weak} + 0.10\,\mathrm{stab} + 0.20\,\mathrm{anti} + 0.15\,\mathrm{subtle}.$$

Here ident is mean score over identity-rewrite probes, $\mathrm{mono} = (1-\rho_{\mathrm{bug}})/2$ uses the pooled bug-level Spearman correlation, and weak is the weak-monotonicity fraction on bug ladders. The latter is the historical field `P3_weak_mono`; it contains no missing-specification measurement. stab is one minus the fraction of scored pairs whose repeated scores cross the artifact-score threshold. anti is the complement of the absolute score correlation with normalized edit distance, clipped to $[0,1]$, and subtle is the clipped mean first-level score drop divided by 0.25. The implementation clips these components to $[0,1]$. These arithmetic indices combine different summaries from $\mathrm{TI}_{\mathrm{var}}$ and do not isolate the effect of changing only the aggregation function.

Geometric comparisons. The two-component comparison in Table 5 is $\mathrm{GM}_{.4,.6} = \mathrm{mono}^{0.4}\mathrm{stab}^{0.6}$, with the same 10^{-6} component floor. Its weights originate in the pass1 search over six components; it is not a fixed equal-weight baseline. The separate cross-pass analysis searches nonnegative weights summing to one over ident, mono, weak, stab, anti, and subtle. Table 9 identifies which pass supplies those fitted weights. Neither procedure fits the weights of $\mathrm{TI}_{\mathrm{var}}$.

C The Earlier 72-Pair Validation Set: Results, Provenance, and Selection Bias

Our earlier version selected and evaluated $\mathrm{TI}_{\mathrm{var}}$ on one 72-pair panel. The numbers below reproduce to three decimals from the frozen score vectors, and we keep them because they motivated the transfer study; the two subsections that follow them record why they are not evidence of transfer.

C.1 THE VALIDATION SET

We evaluated candidate–gold pairs from VERIBENCH (Miranda et al., 2026). One rater supplied holistic ratings on a 0–5 scale in two passes, pass1 under a stricter rubric and pass2 under a more lenient one. The collection covered 75 pairs from 25 problems with three agent generations per problem. One gold filename could not be resolved by the frozen judge pipeline, leaving 72 analyzed pairs from 24 problems. We report each pass and the archived averaged-rating view; they are two ratings by one person, not independent raters. Inter-pass agreement is Spearman 0.249, quadratic-weighted $\kappa = 0.372$, and mean absolute error 0.771 on the shared pairs (Appendix E). This panel is a *validation set*: it informed index-form selection, repeat aggregation, the threshold, and the prompt panel, so correlations on it are optimistic. $\mathrm{TI}_{\mathrm{var}}$ has no fitted weights, so there is no training set, and no analysis on this panel is a test-set result. For each judge, artifact-level agreement is the Spearman correlation between its scores and the human ratings on the analyzed pairs.

C.2 PROPERTY CHECKS ON GPT-5.3-CODEX

Figure 3 summarizes the 24-problem panel for the three semantic Codex prompts. Each bug ladder has five levels, the original artifact plus four cumulative edits, and each artifact has four rewrite probes. Mean scores fall from 1.0 at level zero to 0.24–0.27 after the first cumulative edit and to about zero thereafter. Averages conceal local reversals: the strict prompt reverses on four of 24 ladders, whereas the baseline and checklist prompts have none (Table 6). Renaming, reordering, and style rewrites leave mean scores at or near 1.0. The archived alpha probes are the exception, but their generator can produce malformed Lean text; the baseline and checklist prompts assign them a mean score of zero and the strict prompt 0.179. These historical probes were string edits without Lean elaboration, unlike the probes of Section 5.1; Appendix F records their limitations. The eight-configuration panels contain rewrite probes and bug ladders but no deletion ladders, so they cannot speak to the component that transferred.

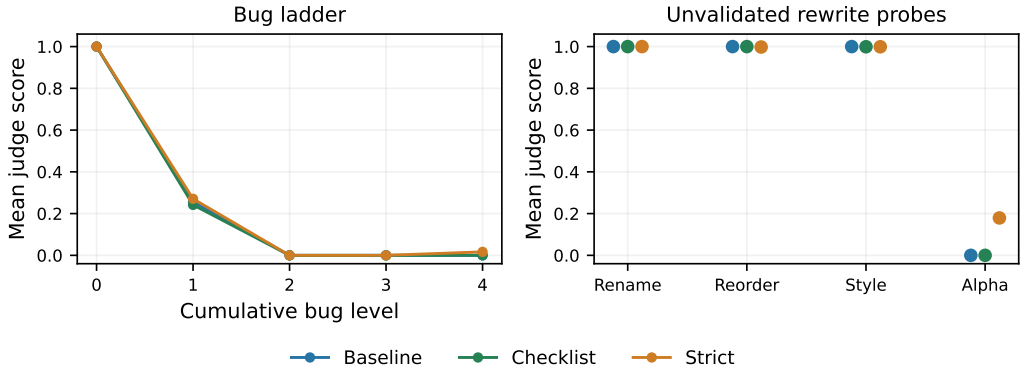


Figure 3: **Early bug edits and malformed alpha probes lower the Codex judges' mean scores.** Each point averages median scores over 24 problems, with three calls per problem and prompt; these are descriptive means, not uncertainty intervals. Local reversals remain visible in Table 6.

C.3 THE INDEX TRACKS JUDGE-LEVEL HUMAN AGREEMENT ON THE VALIDATION SET

The three primary families are GPT-5.3-Codex, Claude Opus 4.7, and Qwen3-Coder. Each eight-configuration panel contains three semantic prompts, a deliberately lexical prompt on the same model, two GPT-4o-mini prompts, edit distance, and a random control. These are family panels with shared controls, not eight independent models per family. Scores are medians over three repeated calls for language-model judges; the Codex-panel controls use nine repeats, and the other panels use three. Figure 4 uses the fixed equal-weight index of Equation 1, normalized within each family panel. Its judge-level Spearman association is positive for each primary family and rating view (Table 5). For Codex, the averaged-rating Pearson and Kendall associations are 0.868 and 0.643. Exact permutation probabilities over all 8! orderings are 0.096, 0.022, and 0.132 for Claude Opus and 0.005, 0.243, and 0.011 for Qwen3-Coder on averaged, pass1, and pass2 ratings (Appendix D.1). Only some cells are

individually distinguishable from a random ordering at the 0.05 level, and the nine tests are unadjusted. Excluding random, edit distance, and the lexical prompt leaves five semantic configurations, with averaged-rating associations of 0.900 for Codex, 0.600 for Claude Opus, and 0.900 for Qwen3-Coder. A separate Leanstral panel has averaged-rating association −0.095 (Appendix E.3), so the positive result did not hold for every recorded family even on this panel. On the held-out judges of Section 5.2, the same index normalized on the train configurations has association −0.332.

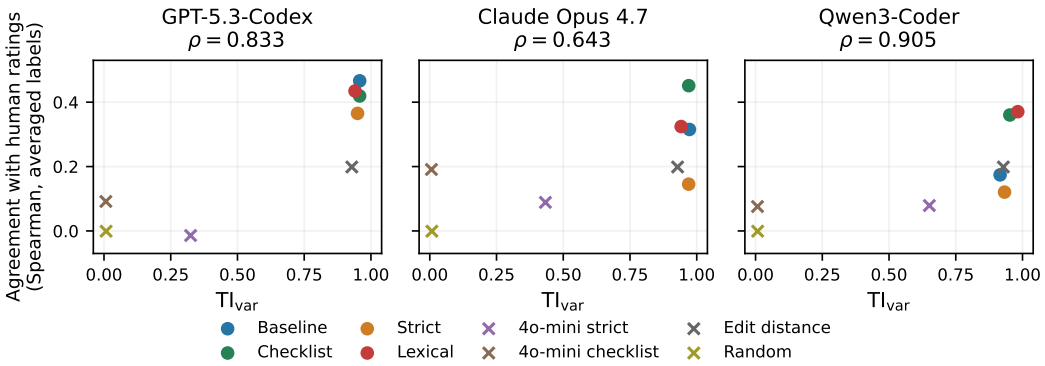


Figure 4: **On the validation set the fixed equal-weight index has a positive association with human agreement in each primary family, but high index values do not rule out lexical judges.** Each point is one configuration; the vertical axis is its Spearman agreement with averaged human ratings on 72 pairs. Panel titles report the correlation across eight configurations; Table 5 gives Codex uncertainty.

	Averaged ratings				$\mathrm{TI}_{\mathrm{var}}$ by pass	
Family	$v1$	$v2$	$\mathrm{GM}_{.4,.6}$	$\mathrm{TI}_{\mathrm{var}}$	pass1	pass2
Codex	0.762	0.476	0.857	0.833	0.905	0.738
Claude Opus	0.429	0.143	0.714	0.643	0.810	0.595
Qwen3-Coder	0.524	0.500	0.786	0.905	0.476	0.857
Codex $\mathrm{TI}_{\mathrm{var}}$: 95% interval				[0.24, 1.00]	[0.41, 1.00]	[0.11, 1.00]
Codex $\mathrm{TI}_{\mathrm{var}}$: permutation p				0.017	0.005	0.045

Table 5: **On the validation set the fixed index is competitive with a two-component geometric comparison whose weights were selected on pass1.** Entries are Spearman correlations between each index and judge-level human agreement, with eight configurations per family. Codex intervals and permutation probabilities use 10,000 resamples and permutations, seed 42; exact permutation probabilities for every cell are in Appendix D.1. Appendix B defines all indices.

C.4 Human agreement alone can miss a bad judge

The lexical prompt, `BAD_STRINGY`, instructs the model to ignore theorem meaning and grade textual overlap. On the Codex panel, its human agreement of 0.435 lies among the semantic prompts' values, 0.365–0.466. A validation protocol that stops at human agreement would accept it. Yet it reverses on 20 of 24 bug ladders (Table 6). The scalar does not expose it either: its index value is 0.940, below the three semantic prompts but far above the failing GPT-4o-mini configurations, and edit distance scores 0.928. In the Qwen panel, the lexical configuration even has the highest index value, 0.982. The failure report therefore carries information that both human agreement and the scalar can miss, which is why the transfer study keeps the per-component report beside any index. The same prompt is the lexical configuration of the transfer study, where it anchors the low end of both axes in Figure 2.

Choice of index, sensitivity, and diagnostics. The two-component geometric comparison in Table 5 uses bug monotonicity and decision stability with weights 0.4 and 0.6 selected on pass1, and

Codex-panel configuration	Human ρ (avg)	$\mathrm{TI}_{\mathrm{var}}$	Ladders with reversals
Baseline prompt	0.466	0.958	0/24
Checklist prompt	0.419	0.958	0/24
Strict prompt	0.365	0.950	4/24
Lexical prompt	0.435	0.940	20/24
GPT-4o-mini, strict	−0.014	0.324	18/24
GPT-4o-mini, checklist	0.092	0.007	16/24
Edit distance	0.199	0.928	24/24
Random	−0.001	0.008	24/24

Table 6: **The lexical judge agrees with the rater despite frequent bug-ladder reversals, and the scalar index penalizes it only slightly.** A reversal is any local score increase as cumulative edits are added; the diagnostic uses the recorded repeat-aggregated scores.

is numerically higher than $\mathrm{TI}_{\mathrm{var}}$ for Codex and Opus on averaged ratings. The bug-monotonicity component alone has averaged-rating associations of 0.643, 0.467, and 0.690, and the full index has higher point correlations in eight of nine family–pass cells, a descriptive comparison on reused ratings (Appendix D.1). A separate search over six component weights gives cross-pass associations of 0.810 and 0.786, and the pass1-selected geometric form reaches 0.769 with a 95% interval of $[0.34, 0.95]$ on an expanded 13-configuration panel (Appendices D and E.2); neither is a held-out estimate. Recomputing the fixed index on a local grid of weight triples gives averaged-rating ranges of 0.738–0.857 for Codex, 0.643–0.714 for Opus, and 0.881–0.976 for Qwen (Appendix D.4). Among the recorded diagnostics, weak monotonicity on bug ladders has the largest association with human agreement on each rating view (Table 7); the archived field name `P3_weak_mono` refers to this bug statistic, not to a deletion property. A historical sweep of the artifact-score threshold from 0.60 to 0.85 lowers the decision-flip rate from 0.058 to 0.046 with overlapping intervals (Table 10); it thresholds $J_\theta(x, g)$, not the judge-level index.

Measured diagnostic	pass1	pass2	avg
Mean identity-rewrite score	−0.21 (0.62)	0.02 (0.98)	−0.02 (0.98)
Bug-rank monotonicity	0.60 (0.13)	0.67 (0.08)	0.71 (0.06)
Bug weak monotonicity	0.63 (0.11)	0.78 (0.03)	0.75 (0.04)
Repeat decision stability	0.47 (0.24)	−0.23 (0.57)	0.14 (0.76)

Table 7: **Bug weak monotonicity has the largest observed association among the measured diagnostics of the validation set; this table contains no deletion-ladder result.** Entries are Spearman correlations with human agreement across eight Codex-panel configurations; unadjusted permutation probabilities appear in parentheses.

C.5 Provenance of the validation set

A zero-call audit traced the panel's ratings and reference files after the earlier version was written. The two rating passes are byte-identical, on the shared items, to the superseded old-prompt rating passes of VeriBench's first participant (75 of 75 and 72 of 75 items), whom VeriBench later re-rated under a corrected prompt together with four more experts; the names of the two passes are swapped relative to the order in which they were rated. The frozen judge pipeline resolved gold files from a copy of VeriBench frozen on 2026-04-12, before the rating window opened, and silently skipped the one task whose gold filename had changed. Of the 24 tasks it scored, only seven have reference bytes identical to what the rater saw at the time the ratings were committed; the other 17 references were rewritten in between. Re-scoring the association on the 21 matched items from those seven tasks against VeriBench's corrected three-expert mean gives 0.333 $[-0.321, 0.543]$ for Codex, -0.381 $[-0.524, 0.657]$ for Claude Opus, and 0.667 $[-0.575, 0.898]$ for Qwen3-Coder, with exact permutation probabilities of 0.428, 0.360, and 0.083 over eight configurations. Under the audit's pre-registered rule, which required one sign in all three families, this is inconclusive; every interval crosses zero, and the matched-item association is not stable to dropping a single configuration. The panel's numbers therefore stand as recorded on the ratings and references the judges saw, with the caveat that those references were not the ones the rater rated for most tasks.

C.6 Selection bias from reusing the validation set

We measured how much choosing among judge configurations on the 72 validation pairs overstates the winner's agreement, using only recorded scores. The pool contains the 109 distinct configurations scored on exactly these pairs, excluding random and edit-distance controls. When ratings are permuted in whole-problem blocks, the best of eight configurations still reaches a mean Spearman agreement of 0.150 (95th percentile 0.317), and the best of all 109 reaches 0.263 (Figure 5a). The observed agreement of 31 configurations exceeds 0.399, the 95th percentile of the maximum over all 109, so the strongest agreements are not selection artifacts. Chosen on 12 problems, the best of eight overstates its agreement on the other 12 problems by 0.153 on averaged ratings, 0.117 on pass1, and 0.194 on pass2, averaged over 4,000 random splits with a Monte Carlo standard error of at most 0.005; individual splits range from -0.334 to $+0.689$ (Figure 5b–c). Beyond eight candidates, choosing among more configurations lowers the held-out agreement of the choice, from 0.335 to 0.296 at 109. A latent-quality simulation fitted to the panel's inter-pass agreement and clustering reproduces the half-panel overstatement within 0.02 and places it at about 0.10 for eight candidates chosen on all 72 pairs. Choosing by a label-free index never overstates agreement by more than 0.01, but its held-out agreement is close to that of a random choice (0.23–0.30 against 0.27–0.28 for two to 32 candidates). At the judge level, taking the maximum over the four reported index forms multiplies the exact two-sided permutation probabilities by 1.9 to 3.2; four of nine family–rating cells remain below 0.05, against six unadjusted. These measurements motivated the split over judge models in Section 4: selection and evaluation must use different judges, and a label-free selector must be shown to pick better judges than chance before it can replace ratings.

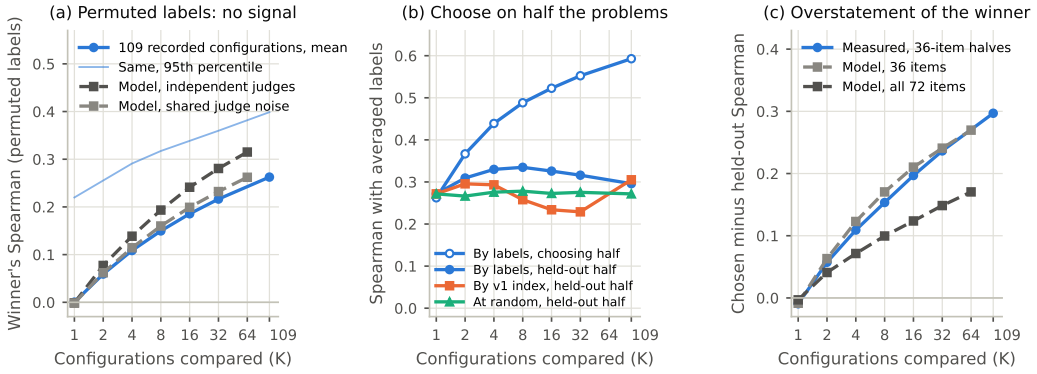


Figure 5: **Choosing among configurations on the validation ratings overstates the winner's agreement.** (a) Largest agreement among K configurations with block-permuted ratings. (b) Agreement of the configuration chosen on 12 problems, on those problems and on the other 12. (c) Chosen-minus-held-out agreement, measured on half panels and simulated for half and full panels; means over 4,000 draws.

D Additional Validation-Set Results

D.1 Post-hoc subset and component diagnostics

Table 8 uses the saved full-precision score vectors, with average ranks for ties and the original eight-configuration normalization fixed. The five semantic configurations exclude random, edit distance, and `BAD_STRINGY`; they retain the target model's baseline, checklist, and strict prompts plus the two GPT-4o-mini prompts. The three-prompt subset retains only those target-model semantic prompts. These are descriptive diagnostics on reused validation ratings, not independent evidence of fine discrimination among useful judges. The component-only column uses C_f, the mean transformed per-file bug correlation, and does not isolate a causal effect of adding penalties. A four-property comparison is unavailable because these panels contain no missing-specification ladders.

The exact tests enumerate all 40,320 labelled orderings of human-agreement values, counting $|\rho|$ at least as large as observed. They differ from the frozen 10,000-permutation Codex procedure

		Full panel ($n = 8$)			Semantic subsets	
Family	Ratings	$\mathrm{TI}_{\mathrm{var}}$	C_f only	Exact p	$n = 5$	$n = 3$
Codex	avg	0.833	0.643	0.015	0.900	1.000
	pass1	0.905	0.690	0.005	0.900	1.000
	pass2	0.738	0.667	0.046	0.800	0.500
Claude Opus	avg	0.643	0.467	0.096	0.600	0.500
	pass1	0.810	0.527	0.022	0.800	0.500
	pass2	0.595	0.575	0.132	0.600	0.500
Qwen3-Coder	avg	0.905	0.690	0.005	0.900	0.500
	pass1	0.476	0.048	0.243	0.400	0.500
	pass2	0.857	0.905	0.011	0.900	0.500
Leanstral	avg	-0.095	-0.310	—	-0.900	-0.500
	pass1	-0.119	-0.524	—	-0.900	-0.500
	pass2	-0.048	-0.143	—	-0.500	0.500

Table 8: Subset associations remain positive for the primary families but negative for Leanstral's five semantic configurations. All association columns are judge-level Spearman correlations with human agreement; the subset columns use $\mathrm{TI}_{\mathrm{var}}$. Exact probabilities are unadjusted across the nine primary-family/pass tests; no additional tests were computed for Leanstral or the subsets.

in Table 5; the corresponding probabilities differ by less than 0.002. The nine probabilities are unadjusted descriptive tests, without correction for protocol selection or reuse of controls. Historical bootstrap intervals for Opus and Qwen remain unavailable; permutation probabilities do not replace them.

D.2 A SEPARATELY WEIGHT-TUNED GEOMETRIC VARIANT

The cross-pass analysis fits a geometric combination of six available components on one rating pass and evaluates it on the other. The selected pass1 weights are 0.4 for bug monotonicity and 0.6 for decision stability. The source searches for weights maximizing judge-level Spearman association on the fitting pass, using four fixed starting vectors and uniform Dirichlet draws on the six-weight simplex. The recorded pass1 solution equals the $(0, 0.4, 0, 0.6, 0, 0)$ starting vector. In the order $(\mathrm{ident}, \mathrm{mono}, \mathrm{weak}, \mathrm{stab}, \mathrm{anti}, \mathrm{subtle})$, the pass2 weights are approximately $(0.167557, 0.037758, 0.170807, 0.601689, 0.007822, 0.014366)$, rounded to six decimals. The saved full-precision weights reproduce the recorded scores; the rounded display is not their exact numerical replacement. The script defaults to seed 42 and 200,000 draws, but the saved fit records do not establish the seed or draw count actually used. Both passes concern the same candidate–gold pairs from one annotator, so the second pass is not a test set. This analysis uses its own frozen rating-alignment output; it is not a re-estimation of the fixed equal-weight index.

Pass supplying weights	Association on that pass	Association on other pass
pass1	0.881	0.810
pass2	0.857	0.786

Table 9: The tuned geometric variant remains positively associated across two rating passes of the same 72 pairs. Entries are judge-level Spearman correlations; the fitted weights are separate from $\mathrm{TI}_{\mathrm{var}}$.

D.3 ARTIFACT-SCORE THRESHOLD

The historical threshold sweep uses the strict Codex prompt on 176 files with three repeated scores, aggregated by the mean rather than the headline panels' median. The sweep covers six thresholds from 0.60 to 0.85 in steps of 0.05. It thresholds artifact scores, not the index used to compare judges. The raw bug-level ranking is independent of this threshold and remains $\rho_{\mathrm{bug}} = -0.790$ throughout the sweep. The intervals in Table 10 describe the recorded sweep; they do not establish accepted-artifact correctness.

Threshold	Flip rate (95% interval)	Pass rate (95% interval)
0.60	0.058 $[0.048, 0.069]$	0.583 $[0.559, 0.608]$
0.70	0.049 $[0.040, 0.059]$	0.559 $[0.537, 0.581]$
0.80	0.051 $[0.042, 0.061]$	0.544 $[0.523, 0.566]$
0.85	0.046 $[0.037, 0.057]$	0.536 $[0.515, 0.557]$

Table 10: Selected points from the historical artifact-score threshold sweep; 0.85 has the lowest observed flip rate. Its interval overlaps those of all other thresholds; this overlap is not a paired test of the difference, and unique optimality has not been established.

D.4 Local weight sensitivity

We recompute the index from committed component vectors on a simplex grid with each weight in $\{0.2, 0.3, 0.4, 0.5\}$ and weights summing to one. This yields 12 triples around the equal-weight choice. The original family-specific normalization and 10^{-6} floor remain fixed. No judge is rescored and no replacement weights are selected from this grid. The script and its complete output are named in the Reproducibility Statement.

Family	pass1 range	pass2 range	avg range
Codex	$[0.810, 0.905]$	$[0.738, 0.810]$	$[0.738, 0.857]$
Claude Opus	$[0.786, 0.810]$	$[0.571, 0.690]$	$[0.643, 0.714]$
Qwen3-Coder	$[0.357, 0.762]$	$[0.738, 0.952]$	$[0.881, 0.976]$

Table 11: The association remains positive on this local weight grid, with the widest pass1 variation for Qwen. Ranges cover judge-level Spearman correlations across the 12 weight triples; they are sensitivity ranges, not confidence intervals.

E Human Agreement and Additional Judge Panels

E.1 Agreement between annotation passes

Table 12 reports the frozen agreement statistics on 72 shared analyzed pairs. The passes reflect different rubric strictness levels from the same annotator, rather than independent draws from a rater population. Their disagreement cannot identify a numerical ceiling on model agreement. The original rubric text remains unavailable.

Statistic	Estimate	95% bootstrap interval
Spearman correlation	0.249	$[-0.015, 0.489]$
Pearson correlation	0.375	$[0.076, 0.604]$
Kendall correlation	0.221	$[-0.016, 0.442]$
Quadratic-weighted κ	0.372	$[0.073, 0.581]$
Mean absolute error	0.771	Not reported

Table 12: The two human passes disagree on the same artifacts, making the choice of rating view consequential. Mean absolute error uses the original 0–5 rating scale; all estimates use 72 overlapping pairs.

E.2 Expanded configuration panel

The expanded panel compares the arithmetic $v2$ with the pass1-selected geometric form $\mathrm{mono}^{0.4}\mathrm{stab}^{0.6}$. Its averaged-rating association is 0.769, with bootstrap interval $[0.34, 0.95]$ (Figure 6). The result concerns 13 configurations on reused validation ratings, not 13 independent model families or unseen data. The plotted geometric index is not $\mathrm{TI}_{\mathrm{var}}$.

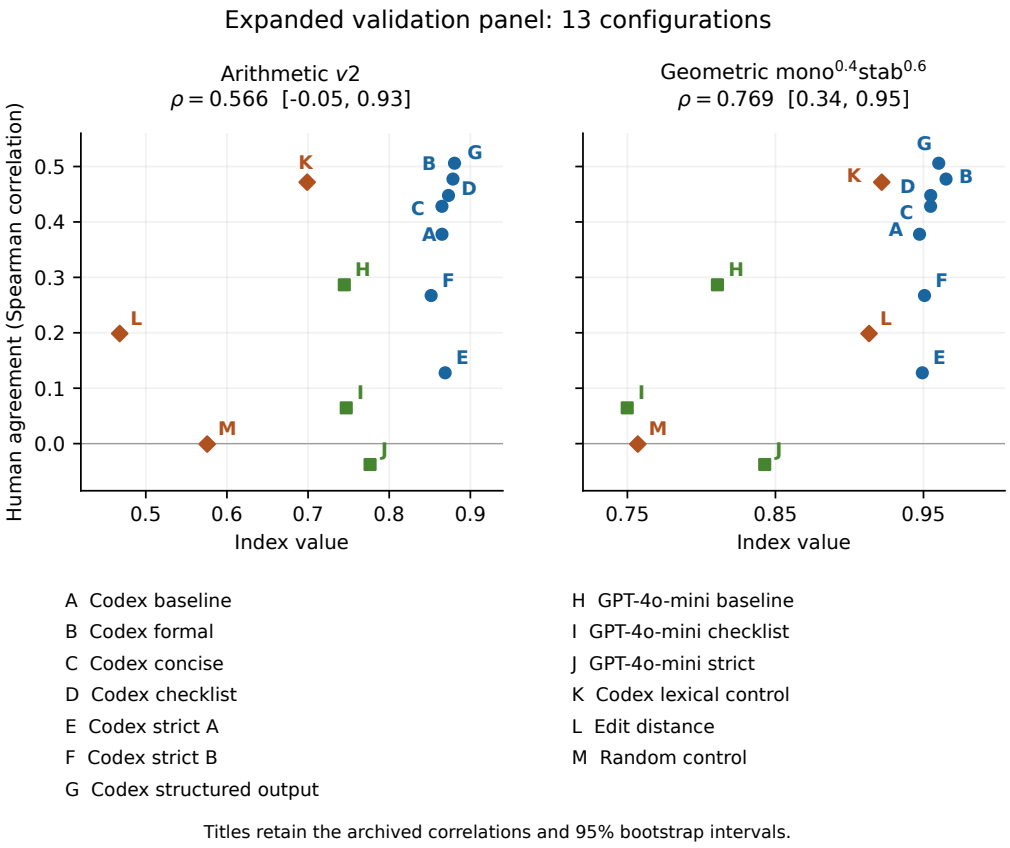


Figure 6: The tuned geometric comparison has a positive association on the expanded validation panel. The two panels compare arithmetic $v2$ with $\mathrm{mono}^{0.4}\mathrm{stab}^{0.6}$ across 13 configurations; the figure retains the archived estimates and intervals.

E.3 Negative results

A separate Leanstral panel yields TI_{var} associations of -0.1190, -0.0476, and -0.0952 on pass1, pass2, and averaged ratings. It uses `labs-leanstral-2603` in the same eight-configuration form, with three recorded scores per row for every configuration and median aggregation. Its five-configuration semantic subset remains negatively associated on all three rating views (Table 8). These recorded results contradict a claim that the fixed index works for every family. They do not establish why the association fails. The different-rubric human pool described in Section 6 also produced no qualifying index in either registered screen. The 32-configuration screen's largest absolute association is negative, so its sign does not support the proposed direction of the screen even before accounting for multiple comparisons.

F Property Diagnostics and Probe Validity

Historical generator limitations. The recorded experiments use string-edit generators without Lean elaboration checks. The alpha routine rewrites names in the theorem signature, leaves proof-body references unchanged, and can duplicate the `:=` marker. Consequently, its low scores cannot establish failure of invariance under valid alpha-renaming. The renaming and reordering routines also lack checks against variable capture and dependent-binder errors. The bug ladder changes relations and connectives, removes a theorem block, and introduces trivial statements; unavailable edits fall back to comments. In each of the Codex, Opus, Qwen, and Leanstral panels, every configuration has 24 ladders with five levels: the extracted reference text and four cumulative edits. Each configuration also has 96 identity-probe rows, comprising four rewrite types per reference. The 24 unedited self-comparisons are recorded at bug-ladder level 0. Compilation of the edited text was not checked, so a score drop can reflect malformed text rather than semantic damage. Thus cumulative edit level is an intended damage order, not a formally verified ordering of semantic quality. These limitations apply to the archived property statistics throughout the paper.

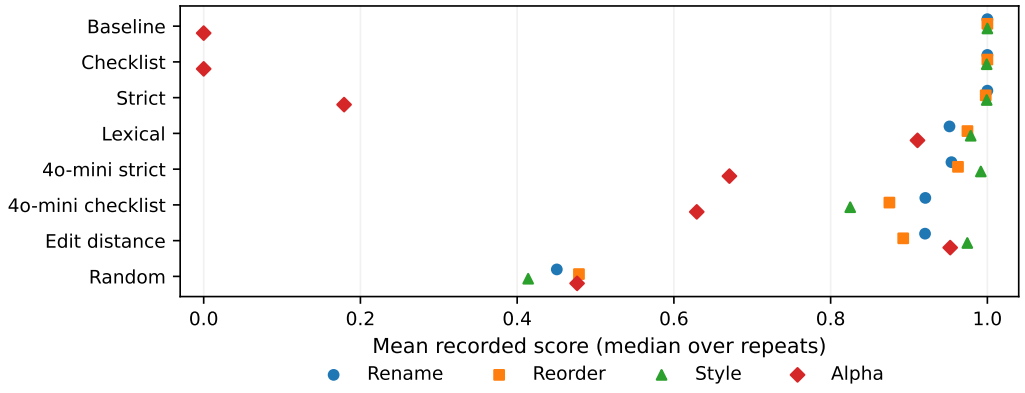


Figure 7: Recorded alpha scores differ sharply from the other rewrite scores for the semantic Codex prompts. Each marker averages repeat-median scores over 24 problems; these unvalidated transformations do not establish semantic sensitivity.

G Unit Tests as Ground Theorems: A Definition We Do Not Adopt

Section 3 follows common formal-methods usage: the formal specification consists of theorem statements about the implementation, and unit tests are not part of it. We record a competing view that we do not adopt. In Lean 4, a unit test is often itself a closed theorem. The gold `myAdd` artifact states its tests as, e.g., `example : myAdd 1 2 = 3 := by native_decide`, an anonymous theorem with no free variables. On this view, a unit test specifies the implementation at one concrete input, and specifications range from such ground instances to universally quantified properties such

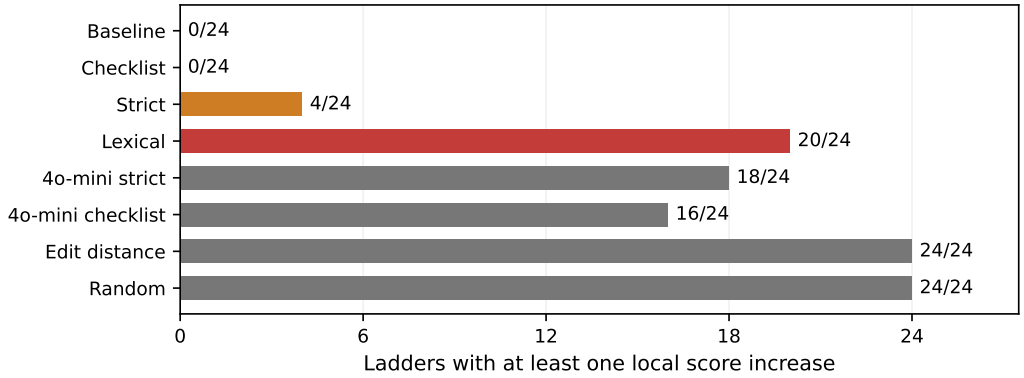


Figure 8: Local increases reveal behavior hidden by the aggregate bug trend. Bars count ladders with at least one increase in the recorded repeat-aggregated score; all configurations have 24 ladders, matching Table 6.

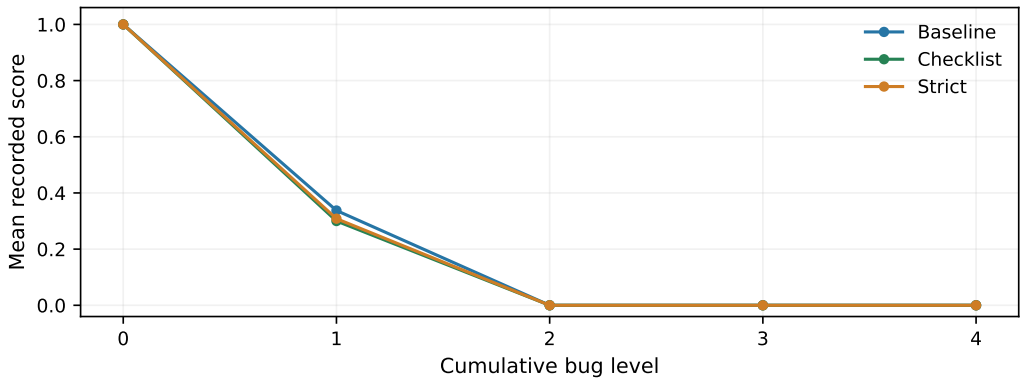


Figure 9: Claude Opus also assigns lower mean scores as historical bug edits accumulate. Each point averages repeat-median scores over 24 problems under the same unvalidated string-edit construction; its trend does not establish semantic validity of every ladder.

as $\forall n,\ \texttt{myAdd}\ n\ 0 = n$. The choice changes terminology, not our measurements. The P3 ladder removes theorem statements and unit tests alike; under the adopted definition, only the removed theorems count as missing specification.